

Sortieren

Selectionsort · Bubblesort · Merge-sort

INF school

0011001110011011000110110100001101111011011110
Florian Köhl

3 Sortieren

- Was heisst Sortieren?
- Was sortieren Menschen?
- Was sortieren Computer?

Sortieren

- Was ist eine aufsteigend sortierte Liste?

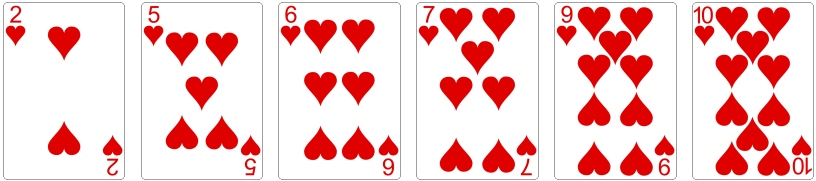
Sortieren

- Was ist eine aufsteigend sortierte Liste?

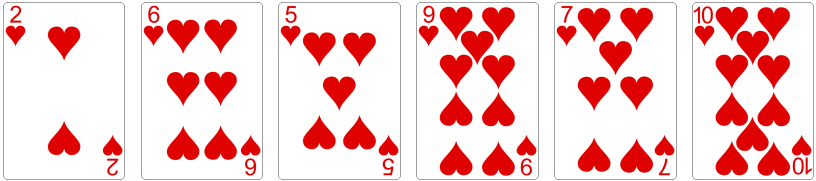
AUFSTEIGEND SORTIERT

Eine Liste ist **aufsteigend sortiert**, sofern jedes Element in der Liste kleiner oder gleich gross ist wie alle nachfolgenden Elemente.

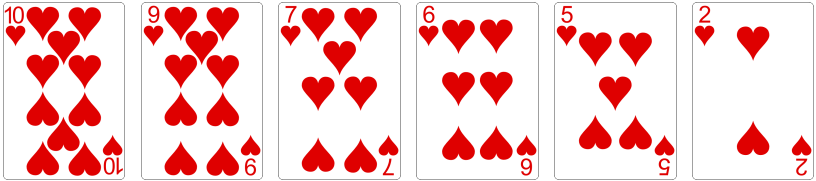
Sortiert?



Sortiert?



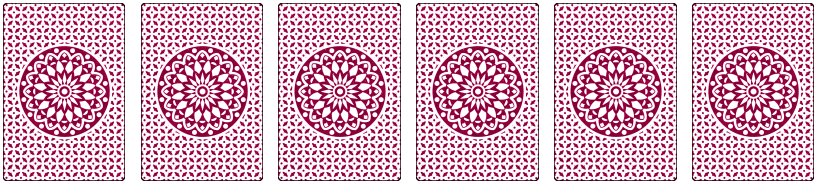
Sortiert?



Sortieralgorithmen

- Bedingungen: Wir dürfen pro Zug …
  - maximal zwei Karten aufgedeckt haben
  - zwei Elemente miteinander vergleichen
  - zwei Elemente miteinander vertauschen
- Entwickelt paarweise ein Vorgehen, um eine Folge von (verdeckten) Karten zu sortieren!

Let’s go!



3.1

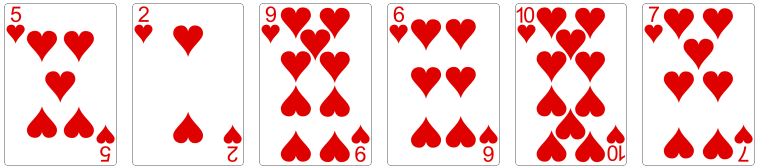
Selectionsort

«Wiederholtes Suchen des Minimums»

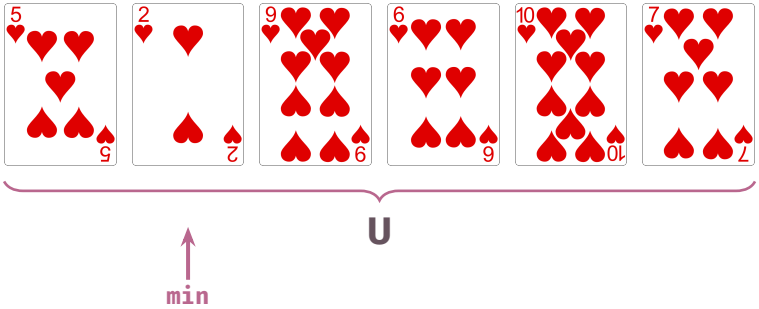
Selectionsort

1. Teile die Elemente in einen sortierten (**S**) und einen unsortierten (**U**) Teil auf. S ist zu Beginn leer, U umfasst alle Elemente.
2. Suche das kleinste Element in U.
3. Tausche es mit dem ersten Element von U, sofern es nicht bereits dort steht.
4. S wird um das eben gefundene Element vergrössert, U wird ein Element kürzer.
5. Falls U mehr als 1 Element hat, gehe zu Schritt 2.
6. Ende: Die Elemente sind sortiert.

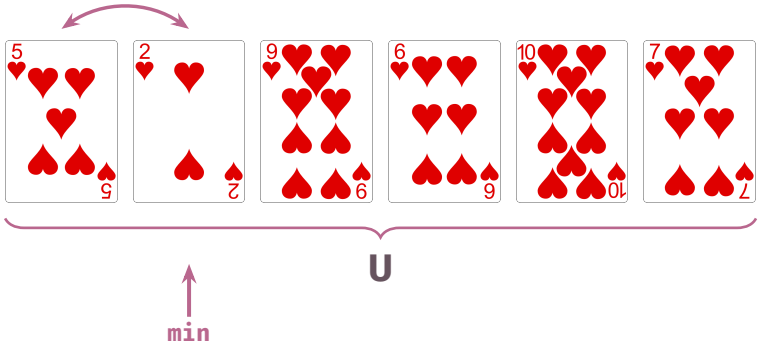
Selectionsort in Aktion



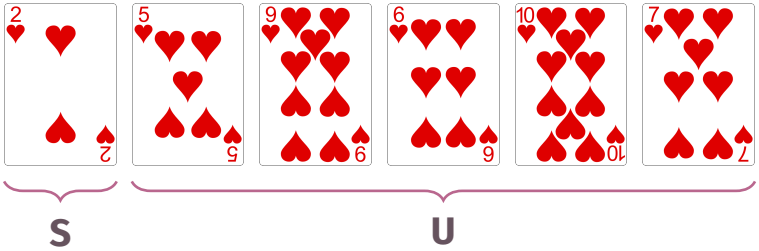
Selectionsort in Aktion



Selectionsort in Aktion

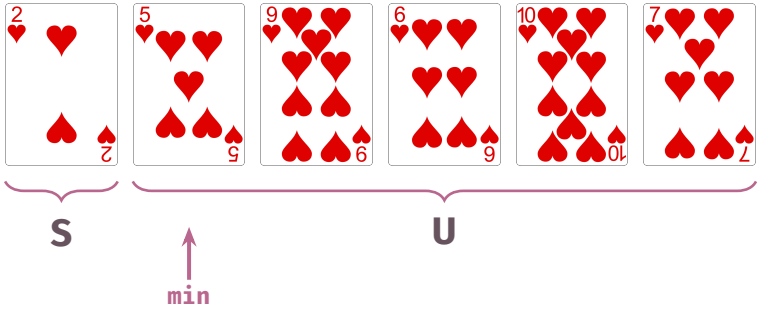


Selectionsort in Aktion



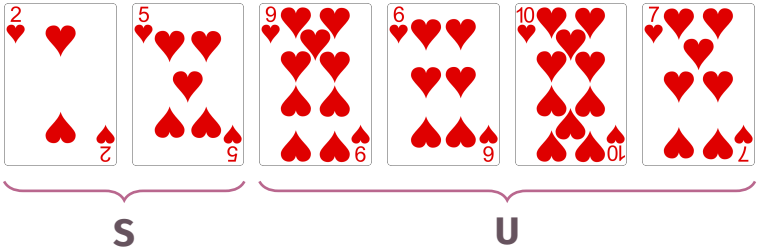
U hat mehr als 1 Element $\rightarrow$ wiederholen

Selectionsort in Aktion



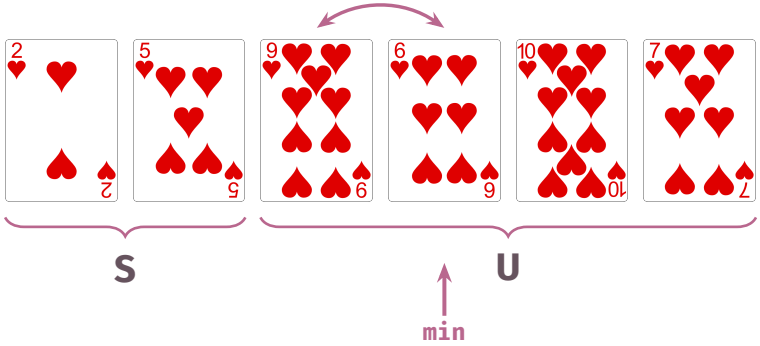
(keine Vertauschung notwendig)

Selectionsort in Aktion

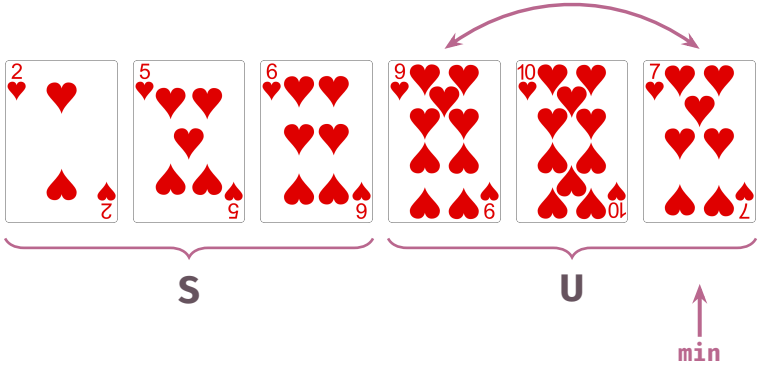


U hat mehr als 1 Element $\rightarrow$ wiederholen …

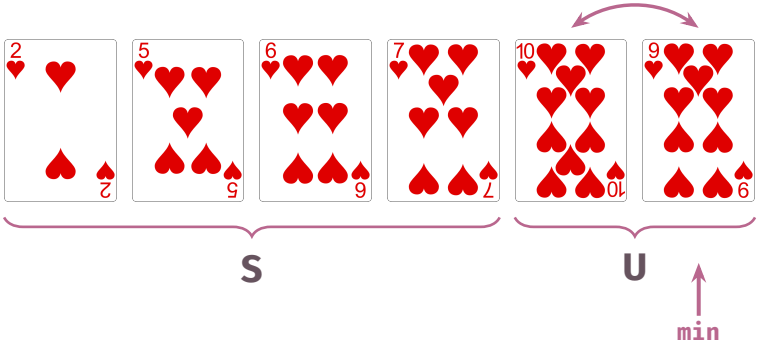
Selectionsort in Aktion



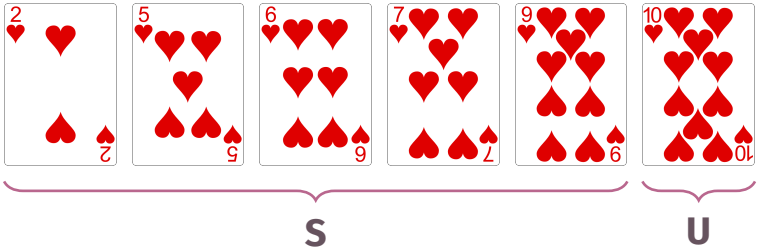
Selectionsort in Aktion



Selectionsort in Aktion



Selectionsort in Aktion



U hat nur 1 Element $\rightarrow$ fertig!

Fragen zu Selectionsort?

Effizienz

- Was ist ein effizienter Algorithmus?

Effizienz: Laufzeit

- Wie «schnell» ist Selectionsort?
- Wie messen wir die Effizienz eines solchen Algorithmus?
  - Zeitmessung?
  - Alternativen?

Effizienz: Laufzeit

- Zeitmessung nicht immer geeignet!
  - Unterschiedliche Hardware
  - Unterschiedliche Implementierung
  - Unterschiedliche Inputs (Eingaben)

Effizienz: Laufzeit

- Zeitmessung nicht immer geeignet!
  - Unterschiedliche Hardware
  - Unterschiedliche Implementierung
  - Unterschiedliche Inputs (Eingaben)
- Grobe Idee: Anzahl **Operationen** zählen
  - «Wie viele Schritte braucht es?»
  - Vergleiche zwischen zwei Elementen
  - Vertauschungen zweier Elemente

Effizienz (Laufzeit)

- **Problem 1:** Wir kennen die Grösse des Inputs nicht.
  - z. B. wie lang die Liste ist, die wir sortieren sollen.
- **Lösung:** Wir geben die Effizienz in Abhängigkeit von der Länge des Inputs an.
  - Oft als *n* bezeichnet.

Effizienz (Laufzeit)

- **Problem 2:** Wir kennen den Input nicht.
  - z. B. wie die Liste aussieht, die wir sortieren sollen.
- **Lösung:** Wir geben die Effizienz für den besten oder schlechtesten Fall an.
  - Oft auch für den «durchschnittlichen» Fall

Beispiel Selectionsort

- Liste mit 6 Elementen
- Betrachte den ersten Durchgang des Algorithmus.
  - Wie viele Vergleiche benötigen wir (bis wir das Minimum gefunden haben)?
  - Wie viele Vertauschungen braucht es dann?
- Wie sieht es in den nächsten Durchgängen aus?

Beispiel Selectionsort

- Liste mit 6 Elementen
  - 1. Durchgang: 5 Vergleiche, max. 1 Vertauschung
  - 2. Durchgang: 4 Vergleiche, max. 1 Vertauschung
  - 3. Durchgang: 3 Vergleiche, max. 1 Vertauschung
  - 4. Durchgang: 2 Vergleiche, max. 1 Vertauschung
  - 5. Durchgang: 1 Vergleich, max. 1 Vertauschung
- $1 + 2 + 3 + 4 + 5 = 15$ Vergleiche, mit max. 5 Vertauschungen.

Beispiel Selectionsort (2)

- Liste mit n Elementen
- Wie viele «Durchgänge» benötigen wir?

Beispiel Selectionsort (2)

- Liste mit n Elementen
  - 1. Durchgang: $n - 1$ Vergleiche, max. 1 Vertauschung
  - 2. Durchgang: $n - 2$ Vergleiche, max. 1 Vertauschung
  - 3. Durchgang: $n - 3$ Vergleiche, max. 1 Vertauschung
  - $\ldots$
  - $(n-1)$. Durchgang: 1 Vergleich, max. 1 Vertauschung
- $1 + 2 + 3 + \cdots + (n-3) + (n-2) + (n-1)$ Vergleiche

$$1 + 2 + 3 + \cdots + (x-2) + (x-1) + x \;=\; \frac{x \cdot (x+1)}{2}$$

$$\sum_{i=1}^{x} i \;=\; \frac{x \cdot (x+1)}{2}$$

Beispiel Selectionsort (2)

- $1 + 2 + 3 + \cdots + (n-3) + (n-2) + (n-1)$ Vergleiche

$$1 + 2 + 3 + \cdots + (n-2) + (n-1) = \frac{(n-1) \cdot n}{2} = \frac{n^2 - n}{2}$$

- $n - 1$ Durchgänge mit je max. 1 Vertauschung: max. $n - 1$ Vertauschungen

Selectionsort: Effizienz

- **Bester Fall:** (Wann?)
  - 0 Vertauschungen
  - $\frac{n^2-n}{2}$ Vergleiche
- **Schlechtester Fall:** (Wann?)
  - $n-1$ Vertauschungen
  - $\frac{n^2-n}{2}$ Vergleiche

3.2

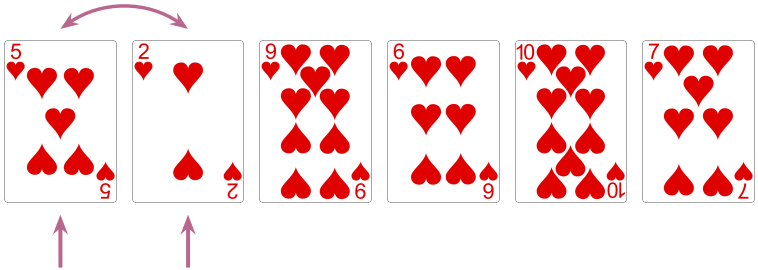
Bubblesort

«Wiederholtes paarweises Vertauschen»

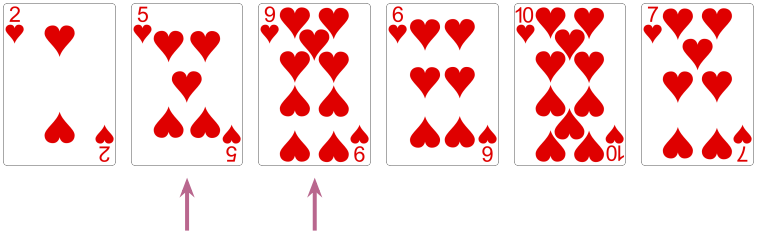
Sortieren: Bubblesort

- Gehe die Liste von links nach rechts durch.
  - Vergleiche jeweils das aktuelle Element mit seinem rechten Nachbarn (falls vorhanden).
  - Falls die beiden Elemente in der falschen Reihenfolge sind, vertausche sie.
- Falls du in diesem Durchgang eine Vertauschung vorgenommen hast, wiederhole den Vorgang.
- Ansonsten ist die Liste sortiert.

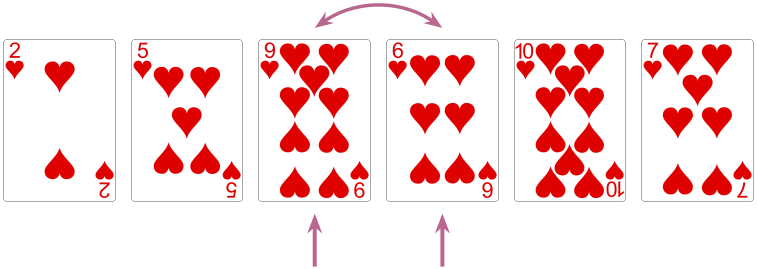
Bubblesort in Aktion



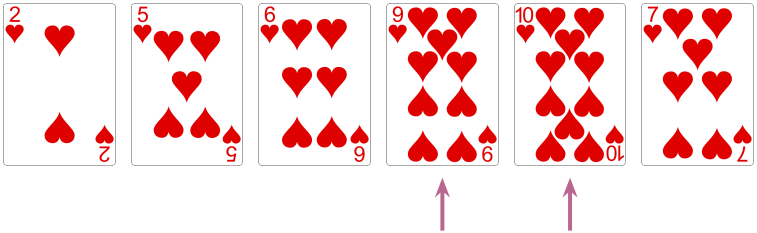
Bubblesort in Aktion



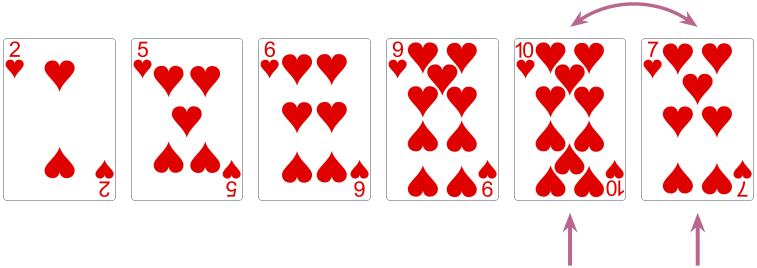
Bubblesort in Aktion



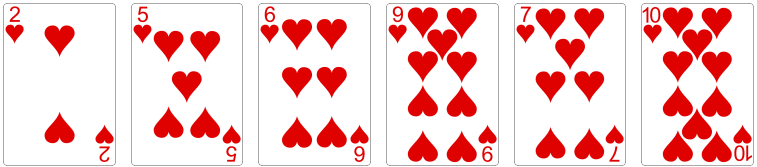
Bubblesort in Aktion



Bubblesort in Aktion



Bubblesort in Aktion



- 1. Durchgang fertig!
- Haben wir vertauscht? Ja $\rightarrow$ also nochmals!

Ihr seid dran!

AUFGABE 38–41: SORTIEREN VON HAND

Löst die Aufgaben 38–41 im Skript.

Zwischenfazit

- Selectionsort und Bubblesort haben beide eine **quadratische Laufzeit**.
  - Die Laufzeitfunktion beinhaltet ein n^2.
  - Verdoppelung der Inputgrösse führt zu Vervierfachung der Laufzeit.
- Wie ineffizient ist das?

«Quadratische Laufzeit»: Beispiel



3.3

Mergesort

«Divide and Conquer»

Divide and Conquer

- Ansatz zum Lösen von Problemen
- **Divide**
  - Aufteilen in Teilprobleme
  - so lange, bis sie trivial lösbar sind
- **Conquer**
  - Kombination der Teillösungen zur Lösung des ganzen Problems

Beispiel: Mergesort

- Effizientes Sortierverfahren
- Basiert auf **Divide and Conquer**

2	8	5	3	9	4	1	7

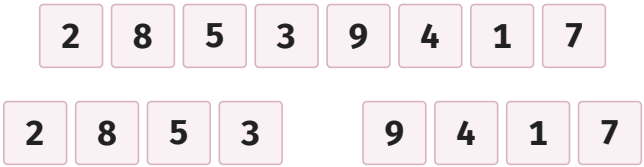
Mergesort: Aufteilen

Wir teilen die Liste in zwei Teillisten auf.

2	8	5	3	9	4	1	7

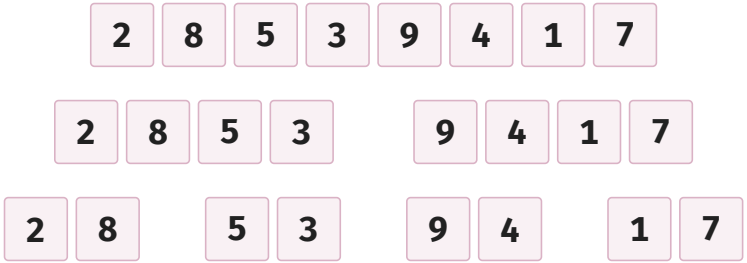
Mergesort: Aufteilen

Wir teilen die Liste in zwei Teillisten auf.



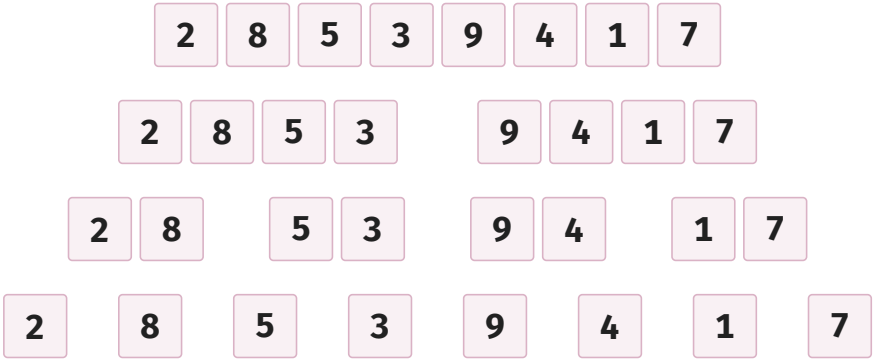
Mergesort: Aufteilen

Gleicher Ansatz für die beiden resultierenden Teillisten.



Mergesort: Aufteilen

Gleicher Ansatz für die beiden resultierenden Teillisten.



Mergesort: Conquer

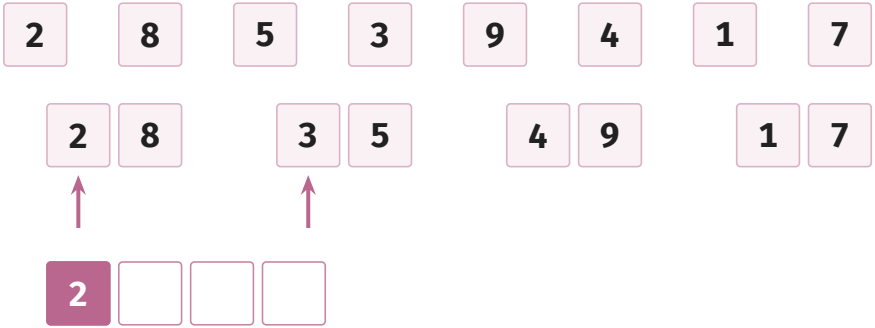
- Resultat: viele kleine Listen der Länge 1
  - Diese sind trivialerweise bereits sortiert!
- Nun müssen wir diese Teillisten wieder zu einem Resultat zusammenfügen.
  - Das «Merge» in Mergesort
  - Wir machen das paarweise.

Mergesort: Merge

«Merge» von zwei sortierten Listen:

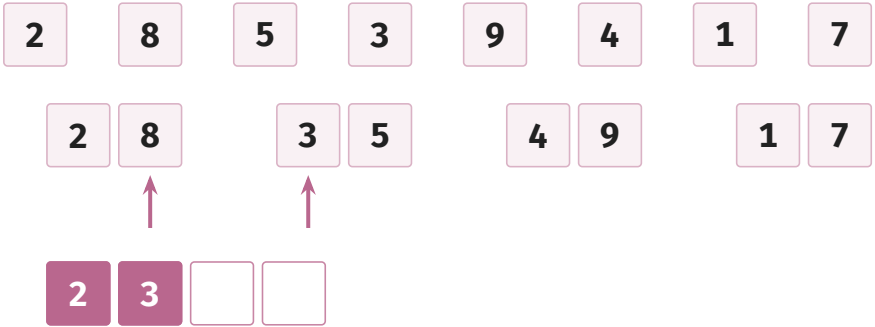
1. Falls beide Listen nicht leer sind:
   - 1.1 Vergleiche das kleinste noch nicht gemergte Element der linken mit demjenigen der rechten Liste.
   - 1.2 Wähle das kleinere der beiden aus und füge es in die neue Liste hinzu.
   - 1.3 Gehe zu Schritt 1.
2. Falls eine der beiden Listen leer ist:
   - 2.1 Füge alle Elemente der nicht leeren Liste der Reihe nach in die neue Liste ein.

Mergesort: Conquer



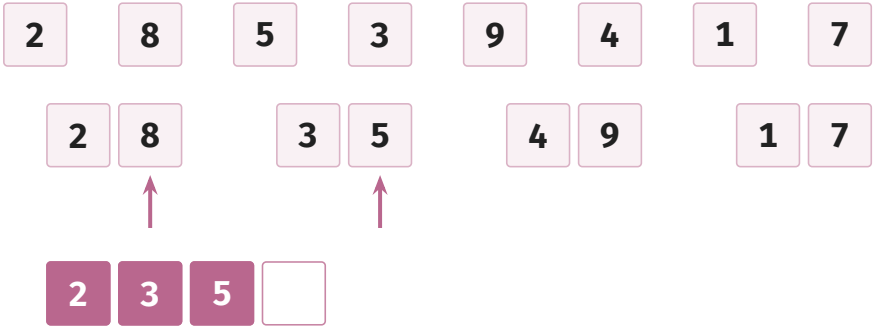
2 < 3: die 2 kommt in die Liste.

Mergesort: Conquer



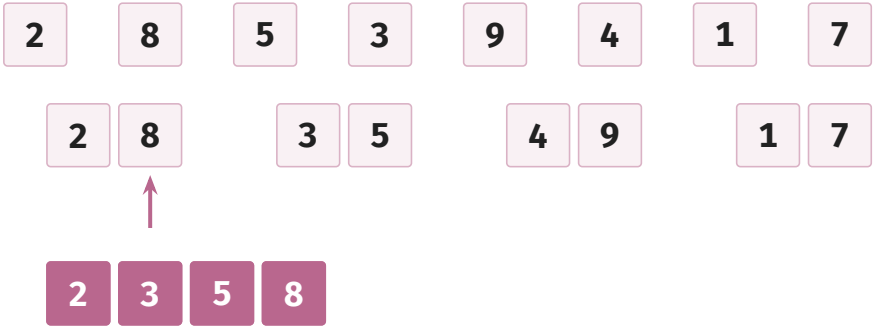
3 < 8: die 3 kommt in die Liste.

Mergesort: Conquer



5 < 8: die 5 kommt in die Liste.

Mergesort: Conquer

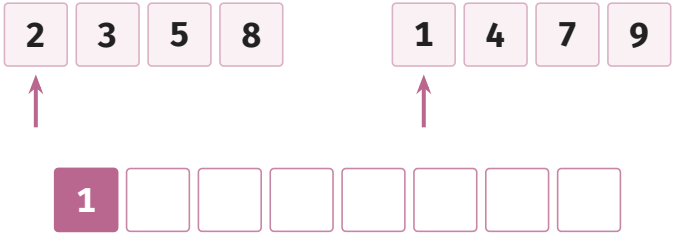


Die rechte Liste ist leer: die 8 kommt in die Liste.

Mergesort: Conquer

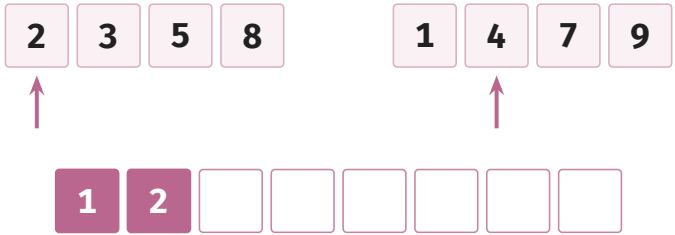
- Gleiches Verfahren mit der rechten Teilliste.
- Dann bleiben noch 2 Teillisten (à je 4 Elemente) übrig.

Mergesort: Conquer



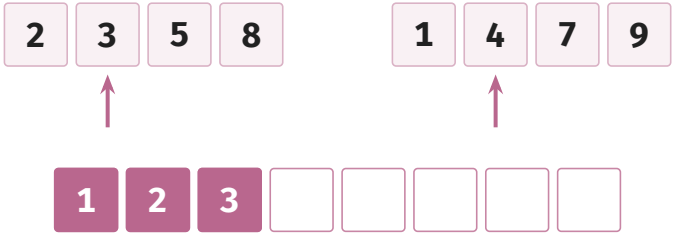
1 < 2: die 1 kommt in die Liste.

Mergesort: Conquer



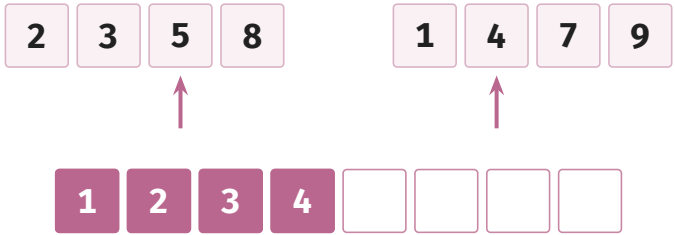
2 < 4: die 2 kommt in die Liste.

Mergesort: Conquer



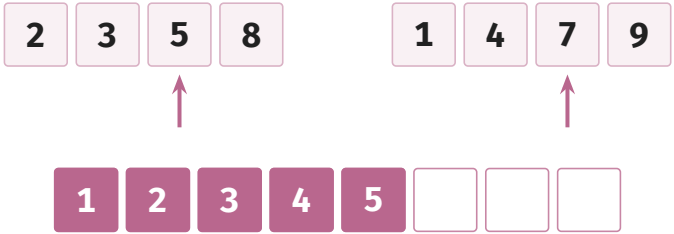
3 < 4: die 3 kommt in die Liste.

Mergesort: Conquer



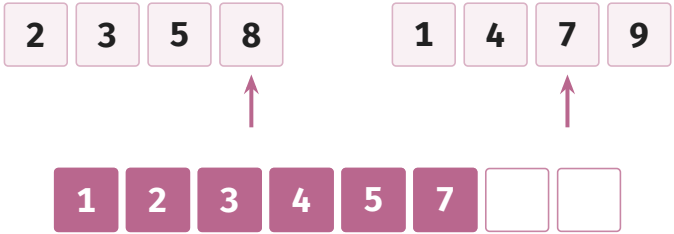
4 < 5: die 4 kommt in die Liste.

Mergesort: Conquer



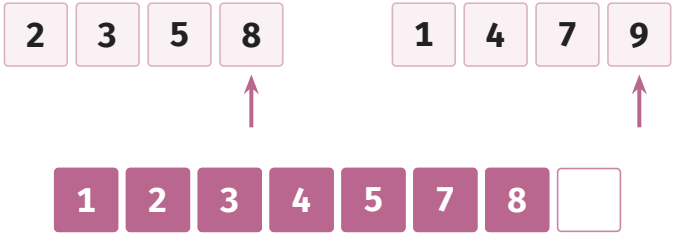
5 < 7: die 5 kommt in die Liste.

Mergesort: Conquer



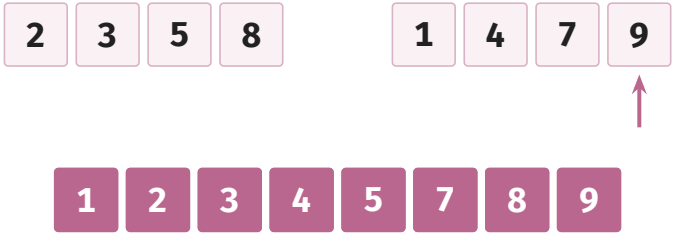
7 < 8: die 7 kommt in die Liste.

Mergesort: Conquer



8 < 9: die 8 kommt in die Liste.

Mergesort: Conquer



Die linke Liste ist leer: die 9 kommt in die Liste.

Mergesort: Effizienz

- $O(n \cdot \log(n))$
- Sprich: Grössenordnung $n \cdot \log(n)$
- Ist das viel besser als «quadratisch»?

n^2 vs. $n \cdot \log(n)$

